GIT & DEVOPS PRACTICAL GUIDE

Experiments 2–8 | Windows | Git Bash, GitHub Desktop and Jenkins

Updated command-by-command guide using the experiment instructions and Python examples from the ABHIJEET-0104/141 GitHub repository.

Source repository: https://github.com/ABHIJEET-0104/141

Use your own GitHub username and repository URL wherever a placeholder appears. Experiments 2–8 are intended to be performed on Windows, as requested.

Experiments	Platform	Main work
2–4	Windows + Git Bash	Branching, pull requests, merge conflicts, .gitignore
5, 7	Windows + GitHub Desktop	GUI branching, fork and pull request
6	Windows + Git Bash	Clone, fork, feature branch and pull request
8	Windows + Jenkins	GitHub-integrated pipeline and automatic build

Before starting

Open Git Bash for command-line Git operations. Use VS Code or Notepad to create/edit program files. Use a web browser for GitHub actions such as creating repositories, forks and pull requests. Experiment 5 and 7 are performed mainly in GitHub Desktop. Experiment 8 uses Jenkins in a browser.

One-time checks (Git Bash)

```
git --version
python --version
git config --global user.name "Omkar-Munde"
git config --global user.email "omkarmunde2121@gmail.com"
git config --global init.defaultBranch main
git config --global --list
```

Important: If your GitHub repository default branch is named master, replace main with master in the relevant commands. Do not blindly mix main and master.

Where to create files

• Create or edit Python files in VS Code/Notepad inside the project folder.

• Run Git commands in Git Bash opened in that same project folder.

• For filenames such as **program2**, save without a .py extension where the experiment explicitly specifies that name. Python can run it using **python program2**.

• When creating .gitignore on Windows, ensure it is named exactly .gitignore and not .gitignore.txt.

Experiment 2 – Git Bash: Even/Odd, Reverse String and Pull Request

Aim: Create a repository, push an even/odd program to main, create GIT-EXAM with a string-reversal program, compare branches and create a pull request from GIT-EXAM to main.

1. Create project and initialize Git (Git Bash)

```
mkdir GitEvenOddProject
cd GitEvenOddProject
git init -b main
```

2. Create an empty GitHub repository

In your browser, GitHub → New repository → name it **GitEvenOddProject**. Create it without README/.gitignore/license so the remote starts empty. Copy the HTTPS URL.

```
git remote add origin https://github.com/YOUR-USERNAME/GitEvenOddProject.git
git remote -v
```

3. Create program1.py (VS Code or Notepad)

```
number = int(input("Enter an integer: "))

if number % 2 == 0:
    print(number, "is an even number")
else:
    print(number, "is an odd number")

python program1.py
```

Test with 8 (even) and 7 (odd).

4. Commit and push to main (Git Bash)

```
git add program1.py
git commit -m "Add even odd checking program"
git push -u origin main
```

5. Create GIT-EXAM branch

```
git switch -c GIT-EXAM
git branch
```

6. Create program2 (without extension) to reverse a string

```
text = input("Enter a string: ")
reversed_text = text[::-1]

print("Original string:", text)
print("Reversed string:", reversed_text)

python program2
```

Example: input hello → output olleh.

7. Commit and push the feature branch

```
git add program2
git commit -m "Add string reverse program"
git push -u origin GIT-EXAM
```

8. Compare branches

```
git branch -a
git log --oneline --graph --decorate --all
git diff main..GIT-EXAM
git diff --name-status main..GIT-EXAM
```

Expected: program2 is an added file on GIT-EXAM; program1.py is already on main.

9. Create pull request (GitHub browser)

• Open your repository → Pull requests → New pull request.

• Base: main. Compare: GIT-EXAM.

• Review the changes; title: Add string reverse program.

• Select Create pull request. Merge only if your practical instructions require it.

Expected result: main contains the even/odd program; GIT-EXAM contains the reverse-string program; the pull request compares GIT-EXAM with main.

Experiment 3 – Git Bash: Branch Pull and Merge Conflict

Aim: Create GIT-EXAM-1 and GIT-EXAM-2, pull changes from the first branch into the second, deliberately create a conflict in program1.py, resolve it, compare branches and create a pull request.

Important: A merge conflict occurs when both branches change the same lines differently. Follow the order below so the conflict is reproducible.

1. Initialize repository and make initial commit

```
mkdir GitConflictProject
cd GitConflictProject
git init -b main
git config user.name "Omkar-Munde"
git config user.email "omkarmunde2121@gmail.com"
git remote add origin https://github.com/YOUR-USERNAME/GitConflictProject.git
echo "Git conflict practical project" > README.md
git add README.md
git commit -m "Initialize project repository"
git push -u origin main
```

First create an empty GitHub repository named GitConflictProject and copy its HTTPS URL into the remote command.

2. Create GIT-EXAM-1 and its two programs

```
git switch -c GIT-EXAM-1
```

Create file **program2** (no extension):

```
text = input("Enter a string: ")
normalized = text.lower().replace(" ", "")

if normalized == normalized[::-1]:
    print("The given string is a palindrome")
else:
    print("The given string is not a palindrome")
```

Create file **program1.py**:

```
print("Squares of numbers from 1 to 10")
for number in range(1, 11):
    print(number, "squared is", number ** 2)
```

```
python program2
python program1.py
git add program1.py program2
git commit -m "Add palindrome and squares programs on GIT-EXAM-1"
git push -u origin GIT-EXAM-1
```

3. Create GIT-EXAM-2 from main and pull GIT-EXAM-1

```
git switch main
git pull origin main
git switch -c GIT-EXAM-2
git pull origin GIT-EXAM-1
ls
git log --oneline --graph --decorate --all
```

4. Add program2.py on GIT-EXAM-2

```
value = input("Enter a word or phrase: ")
clean_value = "".join(value.lower().split())

if clean_value == clean_value[::-1]:
    print("Palindrome")
else:
    print("Not a palindrome")
```

```
python program2.py
```

5. Change program1.py on GIT-EXAM-1 (squares 1 to 5)

```
git switch GIT-EXAM-1
```

Replace the complete content of program1.py with:

```
print("Squares of numbers from 1 to 5")
for number in range(1, 6):
    print(number, "squared is", number ** 2)
```

```
git add program1.py
git commit -m "Change squares range to 1 through 5 on GIT-EXAM-1"
git push origin GIT-EXAM-1
```

6. Make a different change on GIT-EXAM-2 (squares 1 to 15)

```
git switch GIT-EXAM-2
```

Replace the same program1.py lines with:

```
print("Squares of numbers from 1 to 15")
for number in range(1, 16):
    print(number, "squared is", number ** 2)
```

```
git add program1.py program2.py
git commit -m "Add second palindrome program and extend squares range"
git push -u origin GIT-EXAM-2
```

7. Create and resolve the merge conflict

```
git fetch origin
git merge origin/GIT-EXAM-1
git status
```

Open program1.py. It will contain conflict markers similar to the following. Remove all markers and choose the final content:

```
<<<<<<< HEAD
# content from GIT-EXAM-2
=======
# content from GIT-EXAM-1
>>>>>>> origin/GIT-EXAM-1
```

For this practical, use squares from 1 to 10:

```
print("Squares of numbers from 1 to 10")
for number in range(1, 11):
    print(number, "squared is", number ** 2)
```

```
git add program1.py
git commit -m "Resolve conflict in program1.py"
git push origin GIT-EXAM-2
```

8. Compare all three branches

```
git branch -a
git log --oneline --graph --decorate --all
git diff main..GIT-EXAM-1
git diff main..GIT-EXAM-2
git diff GIT-EXAM-1..GIT-EXAM-2
```

9. Create pull request

• GitHub → Pull requests → New pull request.

• Base branch: GIT-EXAM-1. Compare branch: GIT-EXAM-2.

• Review changed files and ensure there are no unresolved conflicts.

• Title: Merge GIT-EXAM-2 into GIT-EXAM-1 → Create pull request.

Expected result: both branches are pushed, a conflict is produced and resolved, all three branches can be compared, and a PR is opened from GIT-EXAM-2 into GIT-EXAM-1.

Experiment 4 – Git Bash: Ignore a Directory with .gitignore

Aim: Push palindrome and squares programs on GIT-EXAM-1, create IGNORE/test1.txt and test2.txt, then exclude the complete IGNORE directory from Git tracking.

1. Create repository and initial commit

```
mkdir GitIgnoreProject
cd GitIgnoreProject
git init -b main
git config user.name "Omkar-Munde"
git config user.email "omkarmunde2121@gmail.com"
git remote add origin https://github.com/YOUR-USERNAME/GitIgnoreProject.git
echo "Git ignore practical project" > README.md
git add README.md
git commit -m "Initialize project repository"
git push -u origin main
```

Create an empty GitHub repository named GitIgnoreProject before connecting the remote.

2. Create GIT-EXAM-1

```
git switch -c GIT-EXAM-1
```

3. Create program2 (palindrome)

```
text = input("Enter a string: ")
clean_text = "".join(text.lower().split())

if clean_text == clean_text[::-1]:
    print("Palindrome")
else:
    print("Not a palindrome")
```

4. Create program1.py (squares 1–10)

```
for number in range(1, 11):
    print(f"Square of {number} = {number ** 2}")
```

```
python program2
python program1.py
git add program1.py program2
git commit -m "Add palindrome and squares programs"
git push -u origin GIT-EXAM-1
```

5. Create IGNORE folder and two text files (Git Bash)

```
mkdir -p IGNORE
echo "This is test file 1" > IGNORE/test1.txt
echo "This is test file 2" > IGNORE/test2.txt
ls IGNORE
```

6. Create .gitignore in the repository root

In VS Code or Notepad, create a file named exactly .gitignore in GitIgnoreProject and enter:

```
# Ignore the complete IGNORE directory at the repository root
/IGNORE/
```

7. Verify ignored files

```
git status --short
git status --ignored --short
git check-ignore -v IGNORE/test1.txt
git check-ignore -v IGNORE/test2.txt
```

Expected: status --ignored shows !! IGNORE/; check-ignore identifies the /IGNORE/ rule.

8. Commit and push only .gitignore

```
git add .gitignore
git commit -m "Add gitignore rule for IGNORE directory"
git push origin GIT-EXAM-1
```

9. Verify on GitHub

Open the repository, select GIT-EXAM-1, and confirm .gitignore, program1.py and program2 are present. IGNORE/, test1.txt and test2.txt must not appear.

Important: If IGNORE was already committed before adding .gitignore, the ignore rule alone does not untrack it. Run: git rm -r --cached IGNORE ; then commit and push.

Experiment 5 – GitHub Desktop: Branching and .gitignore

Aim: Use the GitHub Desktop graphical interface to publish a repository, create GIT-EXAM-1, add palindrome and squares programs, and ignore an IGNORE directory.

1. Create local repository

• Open GitHub Desktop and sign in.

• File → New repository.

• Name: GitHubDesktopIgnoreProject.

• Choose a local path; select Initialize this repository with a README.

• Select Create repository.

2. Publish repository

• Select Publish repository.

• Confirm repository name and visibility (Public/Private as instructed).

• Select Publish repository and verify it on GitHub.

3. Create branch GIT-EXAM-1

• Select Current Branch → New Branch.

• Enter GIT-EXAM-1; use the default branch as the starting point.

• Select Create Branch and check the toolbar.

4. Create program2 and program1.py in the project folder

Use File → Show in Explorer to open the folder. Create **program2** with no extension:

```
text = input("Enter a string: ")
clean_text = "".join(text.lower().split())

if clean_text == clean_text[::-1]:
    print("Palindrome")
else:
    print("Not a palindrome")
```

Create **program1.py**:

```
for number in range(1, 11):
    print(f"Square of {number} = {number ** 2}")
```

Optional test in a terminal opened in the project folder:

```
python program2
python program1.py
```

5. Create IGNORE folder and files

• In File Explorer, create folder IGNORE inside the repository.

• Inside it create test1.txt containing: This is test file 1

• Create test2.txt containing: This is test file 2.

6. Create .gitignore

In the repository root, create a file named exactly .gitignore (not .gitignore.txt) with:

```
/IGNORE/
```

7. Verify changes in GitHub Desktop

• Return to GitHub Desktop → Changes.

• Confirm program1.py, program2 and .gitignore are listed.

• Confirm IGNORE/test1.txt and IGNORE/test2.txt are not listed as normal changes.

• If they appear, verify .gitignore is in the root and the folder name matches.

Optional Git Bash check: Repository → Open in Git Bash, then:

```
git status --ignored --short
git check-ignore -v IGNORE/test1.txt
git check-ignore -v IGNORE/test2.txt
```

8. Commit and push

• In the lower-left commit panel, summary: Add palindrome and squares programs with gitignore.

• Select Commit to GIT-EXAM-1.

• Select Push origin.

• On GitHub, select GIT-EXAM-1 and verify .gitignore/program files exist but IGNORE does not.

Important: If IGNORE was committed earlier, use Git Bash: git rm -r --cached IGNORE, then commit that change and push again.

Experiment 6 – Git Bash: Clone, Fork and Pull Request (AI Project Gallery)

Original repository: https://github.com/KalyanM45/AI-Project-Gallery.git

Aim: Clone the original repository, demonstrate that an account without write permission cannot push to it, fork it, create GIT-EXAM in the fork, contribute a file and open a pull request.

Important: You can create a local branch even without write permission. The restriction is normally on pushing to the original GitHub repository. Do not try to bypass repository permissions.

1. Clone original repository (Git Bash)

```
cd ~/Documents
git clone https://github.com/KalyanM45/AI-Project-Gallery.git
cd AI-Project-Gallery
git remote -v
```

2. Demonstrate push restriction

```
git switch -c GIT-EXAM-TEST
git push -u origin GIT-EXAM-TEST
```

If you do not have write permission, GitHub rejects the push. Capture the error for evidence. Then return to the default branch and delete the temporary local branch:

```
git switch main
git branch -D GIT-EXAM-TEST
```

If the default branch is master, use master. Check with git remote show origin.

3. Fork on GitHub

• Open the original AI-Project-Gallery page in your browser.

• Select Fork → choose your GitHub account → Create fork.

• Open your fork and copy its HTTPS clone URL.

4. Configure original as upstream and fork as origin

```
git remote rename origin upstream
git remote add origin https://github.com/YOUR-USERNAME/AI-Project-Gallery.git
git remote -v
```

Now upstream should point to KalyanM45/AI-Project-Gallery and origin should point to your fork.

5. Synchronize and create feature branch

```
git fetch upstream
git switch main
git pull upstream main
git switch -c GIT-EXAM
```

Replace main with master if the original repository uses master.

6. Add a contribution note (or follow project contribution instructions)

```
mkdir -p contributions
```

Create file contributions/YOUR-GITHUB-USERNAME.md with:

```
# Contribution

This file records my practical-examination contribution to AI Project Gallery.
- Repository reviewed: AI-Project-Gallery
- Feature branch: GIT-EXAM
- Change: Added a contributor note for review.
```

7. Commit and push to your fork

```
git status
```

```
git add contributions/YOUR-GITHUB-USERNAME.md
git commit -m "Add contribution note"
git push -u origin GIT-EXAM
```

8. Create pull request to original repository

• Open your fork on GitHub → Contribute or Compare & pull request.

• Base repository: KalyanM45/AI-Project-Gallery.

• Base branch: original default branch (main/master).

• Head repository: your fork; compare branch: GIT-EXAM.

• Review changes and select Create pull request.

9. Handle review feedback or conflicts

For review feedback, edit the requested file and run:

```
git add <modified-file>
git commit -m "Address pull request review feedback"
git push origin GIT-EXAM
```

If upstream changes must be merged:

```
git fetch upstream
git switch GIT-EXAM
git merge upstream/main
```

Resolve any conflict markers in the affected file, then:

```
git add <resolved-file>
git commit -m "Resolve merge conflict with upstream"
git push origin GIT-EXAM
```

Experiment 7 – GitHub Desktop: Fork and Pull Request (Spam Email Detection)

Original repository: https://github.com/KalyanM45/Spam-Email-Detection.git

Aim: Clone the original repository using GitHub Desktop, demonstrate push restrictions, fork it, create GIT-EXAM in your fork, contribute and open a pull request.

1. Clone original using GitHub Desktop

• Open GitHub Desktop and sign in.

• File → Clone repository → URL tab.

• Paste https://github.com/KalyanM45/Spam-Email-Detection.git.

• Choose a local path → Clone.

2. Demonstrate branch/push restriction

• Current Branch → New Branch → enter GIT-EXAM-TEST → Create Branch.

• Select Publish branch. If you do not have write access, GitHub rejects the push.

• Capture the error; switch back to default branch and delete the temporary branch.

3. Fork the repository

• Open the original Spam-Email-Detection repository in a browser.

• Select Fork → choose your GitHub account → Create fork.

4. Clone your fork separately in GitHub Desktop

• File → Clone repository → URL tab.

• Paste your fork URL, e.g. https://github.com/YOUR-USERNAME/Spam-Email-Detection.git.

• Choose a separate local folder such as Spam-Email-Detection-Fork → Clone.

5. Create GIT-EXAM

• Ensure the fork clone is selected as the current repository.

• Current Branch → New Branch → GIT-EXAM.

• Use the fork default branch as starting point → Create Branch.

6. Add a contribution note

Read the repository README/contribution instructions first. For a simple demonstration, create contributions/YOUR-GITHUB-USERNAME.md:

```
# Contribution

This file records my practical-examination contribution to Spam Email Detection.
- Repository reviewed: Spam-Email-Detection
- Feature branch: GIT-EXAM
- Change: Added a contributor note for review.
```

7. Commit using GitHub Desktop

• Return to GitHub Desktop → Changes.

• Confirm the new file appears and review the diff.

• Summary: Add contribution note.

• Select Commit to GIT-EXAM; verify it appears in History.

8. Publish feature branch

Select Publish branch. Open your fork on GitHub and verify GIT-EXAM exists.

9. Create pull request

• In GitHub Desktop, Branch → Create Pull Request (or use the browser prompt).

• Base repository: KalyanM45/Spam-Email-Detection.

• Base branch: original default branch (main/master).

• Head repository: your fork; compare branch: GIT-EXAM.

• Review changes → Create pull request.

10. Review feedback and conflict handling

For review feedback, edit the file in the fork clone, commit in GitHub Desktop, then Push origin. The existing PR updates automatically.

If an upstream merge is needed, open Repository → Open in Git Bash and run:

```
git remote add upstream https://github.com/KalyanM45/Spam-Email-Detection.git
git fetch upstream
git switch GIT-EXAM
git merge upstream/main
```

If conflicts occur, remove conflict markers and keep the correct combined content, then:

```
git add <resolved-file>
git commit -m "Resolve merge conflict"
git push origin GIT-EXAM
```

Experiment 8 – Jenkins: GitHub-Integrated Pipeline (Windows)

Aim: Install Jenkins locally, connect a Pipeline job to a GitHub repository using a Jenkinsfile, configure Poll SCM and verify that a pushed change triggers a new build.

Important: This experiment is Windows-based. The Jenkinsfile below uses bat commands because the build agent is assumed to be Windows. For a local Jenkins URL (localhost:8080), Poll SCM is the practical trigger; GitHub cannot normally call your computer's localhost webhook.

1. Create GitHub repository and Python file

In GitHub, create a repository named Jenkins-Pipeline-Demo. Clone it or create the files locally, then add program1.py:

```
number = int(input("Enter a number: "))
print("Square:", number ** 2)
```

Commit and push program1.py to the default branch. Copy the repository HTTPS URL.

2. Install and start Jenkins

• Install a Java Development Kit version supported by the Jenkins LTS release you install.

• Install Jenkins using the official Windows installer or your institution's approved method.

• Start the Jenkins service.

• Open a browser and visit http://localhost:8080.

• Retrieve the initial administrator password from the setup wizard's displayed path, enter it, install suggested plugins and create the administrator user.

• Confirm the Jenkins dashboard opens.

3. Install required plugins

• Jenkins dashboard → Manage Jenkins → Plugins.

• Install Git plugin, GitHub plugin and Pipeline plugin (or Pipeline suite).

• Restart Jenkins if requested.

4. Verify Git and Python are available to the Jenkins agent

```
git --version
python --version
```

For Jenkins installed as a Windows service, the service account must be able to access Git and Python through PATH. If not, configure the service account/PATH and restart Jenkins.

5. Add Jenkinsfile to repository root

Create a file named exactly Jenkinsfile (no extension) in Jenkins-Pipeline-Demo:

```
pipeline {
    agent any

    options {
        timestamps()
    }

    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }

        stage('Verify Tools') {
            steps {
```

```
                bat 'git --version'
                bat 'python --version'
            }
        }

        stage('Python Syntax Check') {
            steps {
                bat 'python -m py_compile program1.py'
            }
        }

        stage('Build Complete') {
            steps {
                echo 'Jenkins pipeline completed successfully.'
            }
        }
    }

    post {
        success {
            echo 'Build passed.'
        }
        failure {
            echo 'Build failed. Check the console output.'
        }
    }
}
```

```
git add Jenkinsfile
git commit -m "Add Jenkins pipeline definition"
git push
```

6. Create Pipeline job

• Open http://localhost:8080.

• Select New Item.

• Name: Python-GitHub-Pipeline.

• Select Pipeline → OK.

7. Configure Pipeline script from SCM

• Open job → Configure → Pipeline.

• Definition: Pipeline script from SCM.

• SCM: Git.

• Repository URL: paste the GitHub HTTPS URL.

• Branch specifier: */main (or */master).

• For private repositories, add Jenkins credentials with read access.

• Script Path: Jenkinsfile → Save.

8. Configure Poll SCM

• Open Configure → Build Triggers.

• Select Poll SCM.

• Schedule: H/2 * * * * (checks approximately every two minutes).

• Save.

9. Run first build

• Open the job → Build Now.

• Open Build History → select build number → Console Output.

• Confirm Git checkout, tool verification, Python syntax check and Build Complete succeed.

10. Verify automatic build

• Make a small change to program1.py in your local repository.

• Run git add program1.py, git commit -m "Update Python program", and git push.

• Wait for the next Poll SCM interval, then refresh Jenkins Build History.

• Confirm a new build starts and inspect Console Output.

Important: Do not configure a GitHub webhook to http://localhost:8080/github-webhook/ and expect it to work: localhost is only reachable from your own computer. A webhook requires a secure endpoint reachable from GitHub.

Expected result: Jenkins checks out the GitHub repository, runs the Windows batch commands, validates Python syntax, and starts a new build after a pushed change is detected by Poll SCM.

Final practical checklist

- Experiment 2: program1.py (even/odd), program2 (reverse string), GIT-EXAM branch, comparison and PR.
- Experiment 3: GIT-EXAM-1 and GIT-EXAM-2, pull, conflicting edits, conflict resolution, branch comparison and PR.
- Experiment 4: IGNORE/test1.txt and test2.txt exist locally but are excluded by .gitignore.
- Experiment 5: repeat branching and .gitignore using GitHub Desktop.
- Experiment 6: clone AI-Project-Gallery, fork, push GIT-EXAM to your fork and create PR.
- Experiment 7: repeat fork/PR workflow for Spam-Email-Detection using GitHub Desktop.
- Experiment 8: Jenkins Pipeline job reads Jenkinsfile from GitHub and Poll SCM triggers a build after a push.

Source and reference

Experiment-specific instructions and code were adapted from the public repository below. The source repository contains individual Markdown guides 02.md through 08.md.

https://github.com/ABHIJEET-0104/141

Original source files: 02.md, 03.md, 04.md, 05.md, 06.md, 07.md and 08.md. Always follow your faculty's exact problem statement if it differs from this guide.